

EXECUTIVE SUMMARY

Target: http://combined-test.com
Scan ID: TORTURE-P5-6ac1742b
Scan Date: 2026-03-04 18:05:22
Scan Duration: N/A
Total Requests Sent: 28
Avg Latency: N/A
Peak Concurrency: N/A
Findings: 4 vulnerabilities detected

Severity Breakdown: Critical: 0 | High: 2 | Medium: 2 | Low: 0
AI Inference Calls: 0
LLM Avg Latency: N/A
Circuit Breaker Activations: 0

- Overall Risk Level: High due to presence of two high-severity vulnerabilities in core systems
- Most Critical Category: HIGH - requires urgent patching and mitigation measures immediately
- Immediate Actions: Prioritize remediation for the 2 high-risk vulnerabilities with highest impact on security posture
- Long-Term Recommendations: Implement comprehensive vulnerability management process, prioritize regular scans

EXECUTIVE STRATEGIC IMPACT

```
{
  "Strategic_Attack_Path_Analysis": [
    "An attacker could exploit the OPEN_REDIRECT vulnerability by crafting malicious URLs that redirect users to compromised sites, potentially leading to credential theft or session hijacking.",
    "By chaining this with PATH_TRAVERSAL flaws in web applications, an adversary can access sensitive files and directories on target systems, gaining insight into configuration details and potential vulnerabilities for further exploitation.",
    "Finally, leveraging the OPEN_REDIRECT vulnerability again, the attacker may prompt users through phishing emails to visit these manipulated URLs, thereby achieving a high-impact objective such as full system compromise by compromising user credentials or executing malicious payloads."
  ]
}
```

DETAILED FINDINGS

FILTER: OBJECT REFERENCES (IDOR)

Finding #1: Open Redirect

MEDIUM

CWE: CWE-601
CVSS Score: 5.7 (MEDIUM)

THREAT SCORE: 57/100

Description:

- Vulnerability 'Open Redirect' detected in target endpoint.

Impact:

- Unauthorized access to system resources confirmed.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'OPEN_REDIRECT' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: GET | Param: param_0
URL: http://test-target-torture-.com/api/v1/endpoint_0
Analysis: The application failed to validate and sanitize user input, allowing arbitrary URLs in the redirect.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_0	test_payload_0	Direct reference to target resource.
Access Check	Missing	Application fails to verify authorization.

Result	200 OK	Server returns data for unauthorized request.
--------	--------	---

Payload Specifications:

Vector Category: Open Redirect
Raw Payload: test_payload_0
Encoded: 746573745f7061796c6f61645f30
Encoding Type: None

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_0'
```

HTTP Traffic Snapshot:

Request:

GET http://test-target-torture-.com/api/v1/endpoint_0 HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement strict input validation.

Recommended Code Fix:

```
```python
from flask import Flask, redirect, url_for

app = Flask(__name__)

@app.route('/redirect')
def redirect_to_secure():
 return redirect(url_for('secure_page'))

@app.route('/secure_page')
def secure_page():
 # Secure content handling here
 pass

if __name__ == '__main__':
 app.run(ssl_context='adhoc')
```
```

Finding #2: Path Traversal

HIGH

CWE: CWE-22

CVSS Score: 8.5 (HIGH)

THREAT SCORE: **85/100**

Description:

- The target URL `http://test-target-torture-.com/api/v1/endpoint_1` contains a path traversal vulnerability that allows an attacker to manipulate the request URI, potentially accessing unauthorized resources or injecting malicious payloads.
- The payload `'test_payload_1'` is being sent as part of the HTTP request and may be interpreted by the server in unintended ways if not properly sanitized, increasing the risk of data leakage or injection attacks.
- If exploited, this vulnerability could lead to privilege escalation, information disclosure, or unauthorized access to sensitive resources within the application.

Impact:

- High: The path traversal flaw can be used by malicious actors to bypass authentication controls and gain elevated privileges on the server-side, potentially leading to data theft or system compromise.
- Medium: Unauthorized access to internal resources through manipulated URLs may result in exposure of sensitive information such as user credentials, configuration files, or proprietary data.
- Low/Medium: The vulnerability could enable an attacker to inject malicious payloads into API endpoints, causing unintended behavior, denial-of-service conditions, or even remote code execution if the server processes the input improperly.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern `'PATH_TRAVERSAL'` was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_1

URL: `http://test-target-torture-.com/api/v1/endpoint_1`

Analysis: The application failed to properly validate and sanitize user input, allowing arbitrary path traversal.

Table 1: Payload Decomposition

| Component | Value | Technical Function |
|--------------|----------------|---|
| param_1 | test_payload_1 | Direct reference to target resource. |
| Access Check | Missing | Application fails to verify authorization. |
| Result | 200 OK | Server returns data for unauthorized request. |

Payload Specifications:

Vector Category: Path Traversal
Raw Payload: test_payload_1
Encoded: 746573745f7061796c6f61645f31
Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_1'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_1 HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement strict validation and sanitization of all user-supplied data before processing it in any part of the application. Use a whitelist approach to only allow expected characters and structures.
- Restrict access to sensitive endpoints by enforcing proper authentication, authorization, and role-based permissions at both the server-side and client-side levels.
- Apply input filtering mechanisms such as URL encoding or canonicalization techniques to prevent path traversal attacks. Consider using libraries that provide built-in protection against common injection vectors.

Recommended Code Fix:

```
// Example of a secure code snippet in Python (Flask) to mitigate path traversal
from flask import Flask, request, jsonify
def sanitize_path(path):
    # Remove any directory traversal characters and limit the length
    return path.replace('\', '').replace('/', '').strip()
app = Flask(__name__)
```

```
@app.route('/api/v1/endpoint_1', methods=['GET'])
def endpoint_handler():
    user_input = request.args.get('input')
    safe_input = sanitize_path(user_input)
    # Proceed with processing the sanitized input
    return jsonify({'result': 'Processed safely'}), 200
```

Finding #3: Open Redirect

MEDIUM

CWE: CWE-601
CVSS Score: 5.7 (MEDIUM)

THREAT SCORE: 57/100

Description:

- The target URL `http://test-target-torture-.com/api/v1/endpoint_2` is vulnerable to open redirect attacks.
- An attacker can manipulate the input parameter 'url' to redirect users to malicious sites or internal resources.
- Redirects may bypass security controls and lead to unauthorized access, data leakage, or session hijacking.

Impact:

- User information exposure through redirected links
- Potential privilege escalation by accessing restricted endpoints
- Data theft via compromised redirects

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'OPEN_REDIRECT' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_2
URL: `http://test-target-torture-.com/api/v1/endpoint_2`
Analysis: The application failed to validate and sanitize user input, allowing arbitrary URLs in the redirect.

Table 1: Payload Decomposition

| Component | Value | Technical Function |
|--------------|----------------|--|
| param_2 | test_payload_2 | Direct reference to target resource. |
| Access Check | Missing | Application fails to verify authorization. |

| | | |
|--------|--------|---|
| Result | 200 OK | Server returns data for unauthorized request. |
|--------|--------|---|

Payload Specifications:

Vector Category: Open Redirect
Raw Payload: test_payload_2
Encoded: 746573745f7061796c6f616445f32
Encoding Type: None

Reproduction Command:

```
curl -X POST 'http://test-target-torture-.com/api/v1/endpoint_2'
```

HTTP Traffic Snapshot:

Request:

POST http://test-target-torture-.com/api/v1/endpoint_2 HTTP/1.1
Host: target
{}

Response:

HTTP/1.1 200 OK
Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Validate and sanitize user-supplied input parameters to prevent open redirect manipulation.
- Implement strict content security policies (CSP) to restrict allowed URLs in redirects.
- Use a web application firewall (WAF) or intrusion detection system (IDS) to detect and block suspicious redirect patterns.

Recommended Code Fix:

```
// Example secure code snippet using Python's urllib
import urllib.parse
def sanitize_url(url):
    parsed = urllib.parse.urlparse(url)
    # Strip any potentially malicious segments from the path
    safe_path = '/'.join(parsed.path.split('/')[ :2])
    return urllib.parse.urlunparse((parsed.scheme, parsed.netloc, safe_path, '',
```

FILTER: INJECTION & FUZZING

Finding #4: AI Prompt Injection

CRITICAL

CWE: CWE-77
CVSS Score: 9.0 (CRITICAL)

THREAT SCORE: 90/100

Description:

- The endpoint /api/v1/endpoint_3 is susceptible to prompt injection attacks.
- User input can manipulate the response format and potentially execute malicious code.
- No proper validation or sanitization of user-provided data is implemented.

Impact:

- Sensitive data may be exposed or leaked through manipulated responses.
- Malicious payloads could lead to unauthorized actions, such as account takeover or privilege escalation.
- The vulnerability can cause denial-of-service by overwhelming the server with crafted requests.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'PROMPT_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_3
URL: http://test-target-torture-.com/api/v1/endpoint_3
Analysis: The application failed to properly validate and sanitize user input, allowing malicious payloads to be injected into API requests.

Table 1: Payload Decomposition

| Component | Value | Technical Function |
|--------------|----------------|---|
| param_3 | test_payload_3 | Direct reference to target resource. |
| Access Check | Missing | Application fails to verify authorization. |
| Result | 200 OK | Server returns data for unauthorized request. |

Payload Specifications:

Vector Category: AI Prompt Injection

Raw Payload: test_payload_3

Encoded: 746573745f7061796c6661645f33

Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_3'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_3 HTTP/1.1

Host: target

{}

Response:

HTTP/1.1 200 OK

Content-Type: application/json

{"status": "vulnerable", "data": "revealed"}

Remediation:

- Implement strict input validation and sanitization for all user-provided data.
- Use parameterized queries or prepared statements to prevent injection of malicious code.
- Enforce output encoding and format restrictions on API responses.

Recommended Code Fix:

```
// Example secure code snippet (Java) - replace with appropriate language syntax
public String handleRequest(String userInput) {
    // Validate and sanitize input
    if (!isValidInput(userInput)) {
        throw new IllegalArgumentException("Invalid input");
    }
    // Use parameterized query to prevent injection
    return db.query("SELECT * FROM data WHERE user_id = ?", Arrays.asList(userIn
}
```

SCAN TIMELINE

[Orchestrator] TARGET_ACQUIRED - 2026-03-04 18:05:22

[Sigma] JOB_ASSIGNED - 2026-03-04 18:05:22

[Beta] LOG - 2026-03-04 18:05:23

[Alpha] LOG - 2026-03-04 18:05:23

[Beta] LOG - 2026-03-04 18:05:23

[Beta] LOG - 2026-03-04 18:05:23

[Kappa] LOG - 2026-03-04 18:05:23

[Gamma] LOG - 2026-03-04 18:05:23

[Gamma] LOG - 2026-03-04 18:05:23

[Gamma] LOG - 2026-03-04 18:05:23

[Beta] LOG - 2026-03-04 18:05:23

[Kappa] LOG - 2026-03-04 18:05:24

[Kappa] LOG - 2026-03-04 18:05:24

[Kappa] LOG - 2026-03-04 18:05:24

[Gamma] LOG - 2026-03-04 18:05:24

[Alpha] LOG - 2026-03-04 18:05:24

[Kappa] LOG - 2026-03-04 18:05:24

[Alpha] LOG - 2026-03-04 18:05:24

[Kappa] LOG - 2026-03-04 18:05:24

[Alpha] LOG - 2026-03-04 18:05:24

[Beta] LOG - 2026-03-04 18:05:24

[Alpha] LOG - 2026-03-04 18:05:25

[Gamma] VULN_CONFIRMED - 2026-03-04 18:05:24

[Gamma] VULN_CONFIRMED - 2026-03-04 18:05:25

[Gamma] VULN_CONFIRMED - 2026-03-04 18:05:25

[Gamma] VULN_CONFIRMED - 2026-03-04 18:05:26

[Gamma] VULN_CONFIRMED - 2026-03-04 18:05:26

[Kappa] GI5_LOG - 2026-03-04 18:05:32